第 10 讲

后端 2

主要目标

1. 理解滑动窗口优化。
2. 理解位姿图优化。
3. 理解带 IMU 紧耦合的优化。
4. 通过实验掌握 g2o 的位姿图。

第 9 讲我们重点介绍了以 BA 为主的图优化。BA 能精确地优化每个相机位姿与特征点位置。不过在更大的场景中，大量特征点的存在会严重降低计算效率，导致计算量越来越大，以至于无法实时化。本讲的第一部分将介绍一种简化的 BA：位姿图。

10.1 滑动窗口滤波和优化

10.1.1 实际环境下的 BA 结构

带有相机位姿和空间点的图优化称为 BA，它能够有效地求解大规模的定位与建图问题。这在 SfM 问题中十分有用，但是在 SLAM 过程中，我们往往需要控制 BA 的规模，保持计算的实时性。倘若计算能力无限，那不妨每时每刻都计算整个 BA——但是那不符合现实需要。现实条件是，我们必须限制后端的计算时间，比如 BA 规模不能超过 1 万个路标点，迭代不超过 20 次，用时不超过 0.5 秒，等等。像 SfM 那样用一周时间重建一个城市地图的算法，在 SLAM 里不见得有效。

控制计算规模的做法有很多，比如从连续的视频中抽出一部分作为**关键帧**[89]，仅构造关键帧与路标点之间的 BA，于是非关键帧只用于定位，对建图则没有贡献。即便如此，随着时间的流逝，关键帧数量会越来越多，地图规模也将不断增长。像 BA 这样的批量优化方法，计算效率会（令人担忧地）不断下降。为了避免这种情况，我们需要用一定手段控制后端 BA 的规模。这些手段可以是理论上的，也可以是工程上的。

例如，最简单的控制 BA 规模的思路，是仅保留离当前时刻最近的 N 个关键帧，去掉时间上更早的关键帧。于是，我们的 BA 将被固定在一个时间窗口内，离开这个窗口的则被丢弃。这种方法称为滑动窗口法[91]。当然，取这 N 个关键帧的具体方法可以有一些改变，例如，不见得必须取时间上最近的，而可以按照某种原则，取时间上靠近，空间上又可以展开的关键帧，从而保证相机即使在停止不动时，BA 的结构也不至于缩成一团（这容易导致一些糟糕的退化情况）。如果我们在帧与帧的结构上再考虑得深一些，也可以像 ORB-SLAM2[88] 那样，定义一种称为“共视图”（Covisibility graph）的结构（如图 10-1 所示）。所谓共视图，就是指那些“与现在的相机存在共同观测的关键帧构成的图）。于是，在 BA 优化时，我们按照某些原则在共视图内取一些关键帧和路标进行优化，例如，仅优化与当前帧有 20 个以上共视路标的关键帧，其余部分固定不变。当共视图关系能够正确构造的时候，基于共视图的优化也会在更长时间内保持最优。

滑动窗口也好，共视图也好，大体而言，都是我们对实时计算的某种工程上的折中。不过在理论上，它们也引入了一个新问题：刚才我们谈到要“丢弃”滑动窗口之外，或者“固定”共视图之外的变量，这个“丢弃”和“固定”具体怎样操作呢？“固定”似乎很容易理解，我们只需将共视图之外的关键帧估计值保持不变即可。但是“丢弃”，是指完全弃置不用，即窗口外的变量完全不对窗口内的变量产生任何影响，还是说窗口外的数据**应该**对窗口内的有一些影响，但实际上被我们忽略了？如果有影响，这种影响应该是什么样子？它够不够明显，能不能忽略？

接下来，我们就要谈谈这些问题。它们在理论上应该如何处理，以及在工程上能不能做一些简化手段。

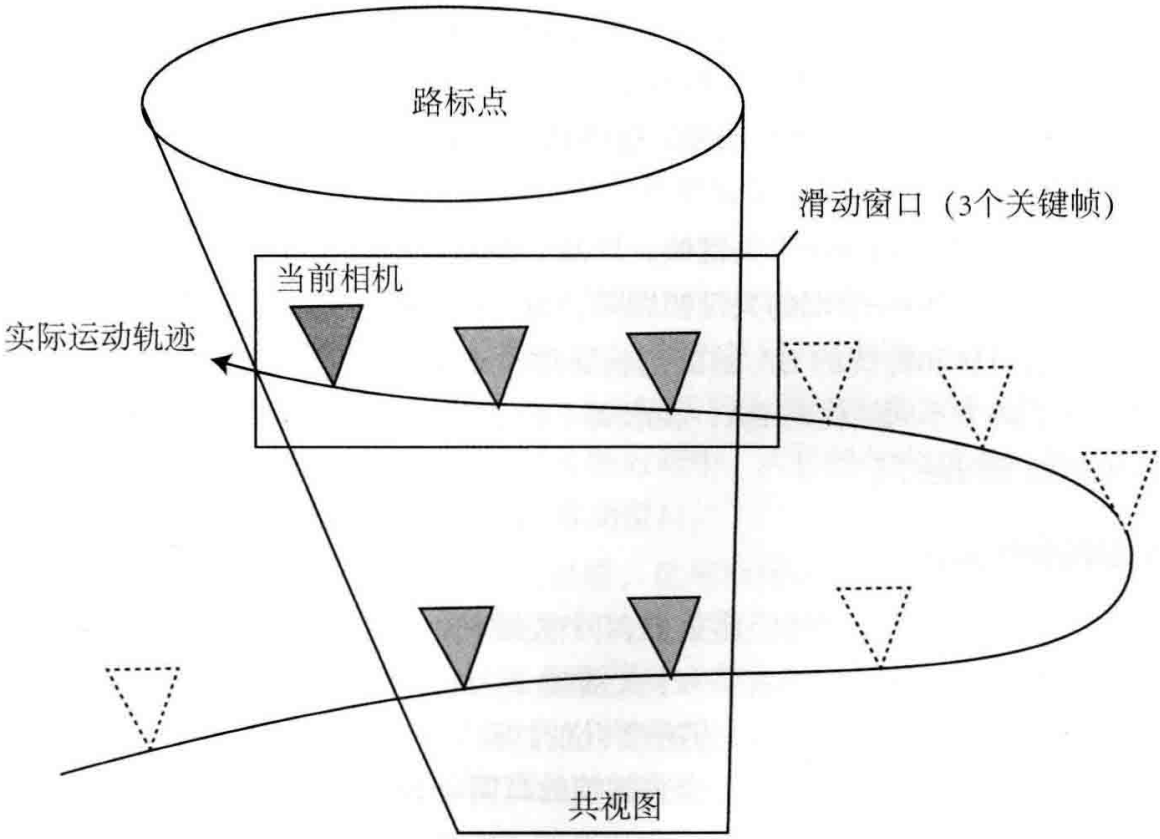

图 10-1　滑动窗口和共视图的示意图

10.1.2　滑动窗口法

现在考虑一个滑动窗口。假设这个窗口内有 N 个关键帧，它们的位姿表达为：

$$\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N,$$

我们假设它们在向量空间，即用李代数表达，那么，关于这几个关键帧，我们能谈论些什么呢？

显然，我们关心这几个关键帧的位置在哪里，以及它们的不确定度如何，这对应着它们在高斯分布假设下的均值协方差。如果这几个关键帧还对应着一个局部地图，则我们可以顺带着问整个局部系统的均值和方差应该是多少。设这个滑动窗口中还有 M 个路标点：$\boldsymbol{y}_1, \ldots, \boldsymbol{y}_M$，它们与 N 个关键帧组成了局部地图。显然我们可以用第 9 讲介绍的 BA 方法处理这个滑动窗口，包括建立图优化模型，构建整体的 Hessian 矩阵，然后边缘化所有路标点来加速求解。在边缘化时，我们考虑关键帧的位姿，即

$$[\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N]^{\mathrm{T}} \sim N([\boldsymbol{\mu}_1, \ldots, \boldsymbol{\mu}_N]^{\mathrm{T}}, \boldsymbol{\Sigma}).$$

其中 $\boldsymbol{\mu}_k$ 为第 k 个关键帧的位姿均值，$\boldsymbol{\Sigma}$ 为所有关键帧的协方差矩阵，那么显然，均值部分就是

指 BA 迭代之后的结果，而 $\boldsymbol{\Sigma}$ 就是对整个 BA 的 $\boldsymbol{H}$ 矩阵进行边缘化之后的结果，即第 9 讲提到的矩阵 $\boldsymbol{S}$。我们认为读者已经熟悉这个流程了。

在滑动窗口中，当窗口结构发生改变，这些状态变量应该如何变化？这件事情可以分成两部分讨论：

1. 我们需要在窗口中新增一个关键帧，以及它观测到的路标点。
2. 我们需要把窗口中一个旧的关键帧删除，也可能删除它观测到的路标点。

这时，滑动窗口法和传统的 BA 的区别就显现出来了。显然，如果按照传统的 BA 来处理，那么这仅仅对应于两个不同结构的 BA，在求解上没有任何差别。但在滑动窗口的情况下，我们就要讨论具体的细节问题了。

新增一个关键帧和路标点

考虑在上个时刻，滑动窗口已经建立了 N 个关键帧，我们也已知道它们服从某个高斯分布，其均值和方差如前所述。此时，新来了一个关键帧 $\boldsymbol{x}_{N+1}$，那么整个问题中的变量变为 $N+1$ 个关键帧和更多路标点的集合。这实际上仍是平凡的，我们只需按照正常的 BA 流程处理即可。对所有点进行边缘化时，即得到这 $N+1$ 个关键帧的高斯分布参数。

删除一个旧的关键帧

当考虑删除旧关键帧时，一个理论问题将显现出来。例如我们要删除旧关键帧 $\boldsymbol{x}_1$，但是 $\boldsymbol{x}_1$ 并不是孤立的，它会和其他帧观测到同样的路标。将 $\boldsymbol{x}_1$ 边缘化之后将导致整个问题不再稀疏。和第 9 讲一样，我们举一个示意图，如图 10-2 所示。

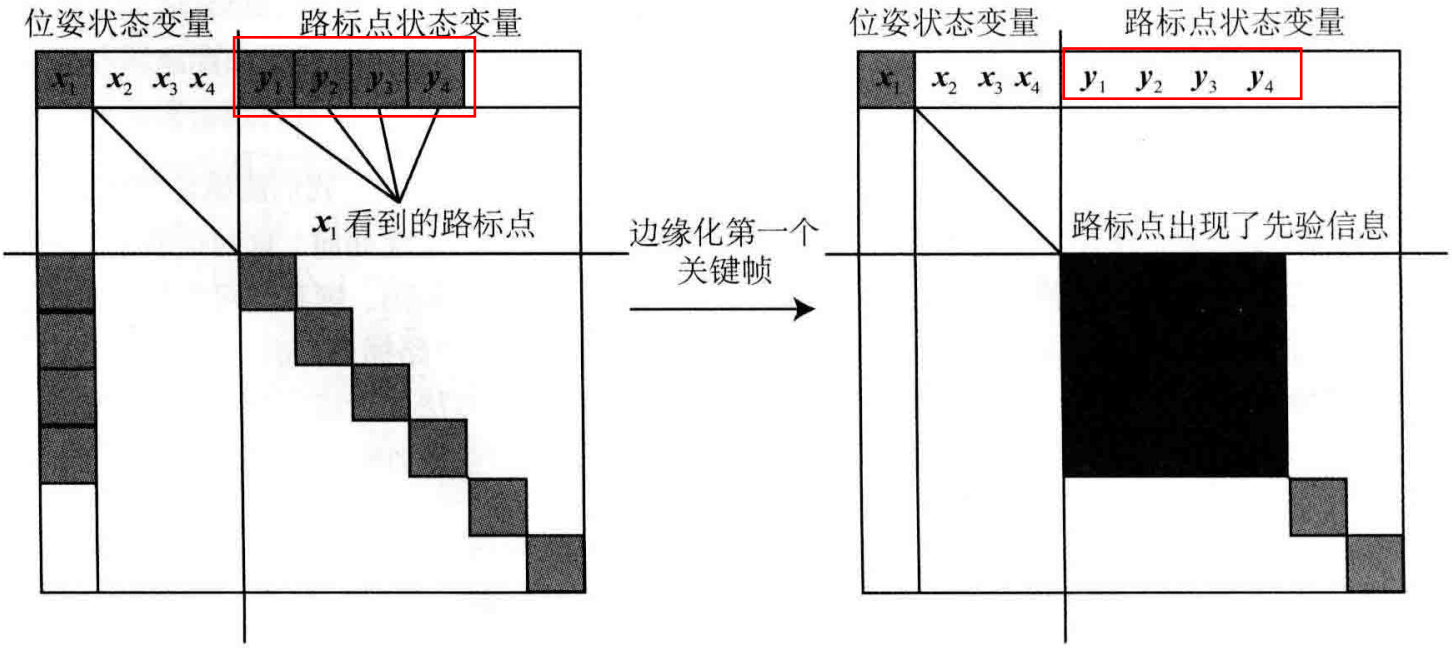

图 10-2 滑动窗口删除关键帧将破坏路标部分的对角块结构

在这个例子中，我们假设 $\boldsymbol{x}_1$ 看到了路标点 $\boldsymbol{y}_1$ 至 $\boldsymbol{y}_4$，于是，在处理之前，BA 问题的 Hessian 矩阵应该像图 10-2 中的左图一样，在 $\boldsymbol{x}_1$ 行的 $\boldsymbol{y}_1$ 到 $\boldsymbol{y}_4$ 列存在着非零块，表示 $\boldsymbol{x}_1$ 看到了它们。这时考虑边缘化 $\boldsymbol{x}_1$，那么 Schur 消元过程相当于通过矩阵行和列操作消去非对角线处几个非零矩阵块，显然这将导致右下角的路标点矩阵块不再是非对角矩阵。这个过程称为边缘化中的**填入**（Fill-in）[91]。

回顾第 9 讲中介绍的边缘化，当我们边缘化路标点时，Fill-in 将出现在左上角的位姿块中。不过，因为 BA 不要求位姿块为对角块，所以稀疏 BA 求解仍然可行。但是，当边缘化关键帧时，将破坏右下角路标点之间的对角块结构，这时 BA 就无法按照先前的稀疏方式迭代求解。这显然是个十分糟糕的问题。实际上，在早期的 EKF 滤波器后端中，人们确实保持着一个稠密的 Hessian 矩阵，这也使得 EKF 后端没法处理较大规模的滑动窗口。

不过，如果我们对边缘化的过程进行一些改造，也可以保持滑动窗口 BA 的稀疏性。例如，在边缘化某个旧的关键帧时，同时边缘化它观测到的路标点。这样，路标点的信息就会转换成剩下那些关键帧之间的共视信息，从而保持右下角部分的对角块结构。在某些 SLAM 框架[70, 89] 中，边缘化策略会更复杂。例如在 OKVIS 中，我们会判断要边缘化的那个关键帧，它看到的路标点是否在最新的关键帧中仍能看到。如果不能，就直接边缘化这个路标点；如果能，就丢弃被边缘化关键帧对这个路标点的观测，从而保持 BA 的稀疏性。

SWF 中边缘化的直观解释

我们知道边缘化在概率上的意义就是指条件概率。所以，当我们边缘化某个关键帧，即“保持这个关键帧当前的估计值，求其他状态变量以这个关键帧为条件的条件概率”。所以，当某个关键帧被边缘化，它观测到的路标点就会产生一个“**这些路标应该在哪里**”的先验信息，从而影响其余部分的估计值。如果再边缘化这些路标点，那么它们的观测者将得到一个“**观测它们的关键帧应该在哪里**”的先验信息。

从数学上看，当我们边缘化某个关键帧，整个窗口中的状态变量的描述方式，将从联合分布变成一个条件概率分布。以上面的例子来看，就是说：

$$p(\boldsymbol{x}_1, \ldots \boldsymbol{x}_4, \boldsymbol{y}_1, \ldots \boldsymbol{y}_6) = p(\boldsymbol{x}_2, \ldots, \boldsymbol{x}_4, \boldsymbol{y}_1, \ldots \boldsymbol{y}_6 | \boldsymbol{x}_1) \underbrace{p(\boldsymbol{x}_1)}_{\text{舍去}}. \tag{10.1}$$

然后舍去被边缘化部分的信息。在变量被边缘化之后，我们在工程中就不应再使用它。所以滑动窗口法比较适合 VO 系统，而不适合大规模建图的系统。

由于现在 g2o 和 Ceres 还未直接支持滑动窗口法中的边缘化操作①，我们略去本节对应的实验部分。希望理论部分可以帮助读者理解基于滑动窗口的 SLAM 系统。

①工程中我们可以通过某些巧妙的手段绕过 g2o 和 Ceres 的框架限制，但这往往非常烦琐，不适合在书中演示。

10.2 位姿图

10.2.1 位姿图的意义

根据前面的讨论，我们发现特征点在优化问题中占据了绝大部分。实际上，经过若干次观测之后，收敛的特征点位置变化很小，发散的外点则已被剔除。对收敛点再进行优化，似乎是有些费力不讨好的。因此，我们更倾向于在优化几次之后就把特征点固定住，只把它们看作位姿估计的约束，而不再实际地优化它们的位置估计。

沿着这个思路继续思考，我们会想到：是否能够完全不管路标而只管轨迹呢？我们完全可以构建一个只有轨迹的图优化，而位姿节点之间的边，可以由两个关键帧之间通过特征匹配之后得到的运动估计来给定初始值。不同的是，一旦初始估计完成，我们就不再优化那些路标点的位置，而只关心所有的相机位姿之间的联系。通过这种方式，我们省去了大量的特征点优化的计算，只保留了关键帧的轨迹，从而构建了所谓的位姿图（Pose Graph），如图 10-3 所示。

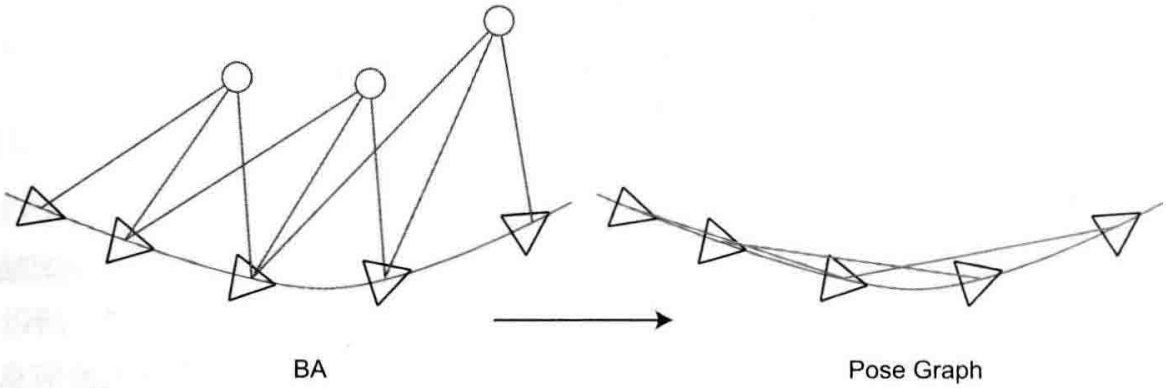

图 10-3 位姿图示意图。当我们不再优化 BA 中的路标点，仅把它们看成对姿态节点的约束时，就得到了一个计算规模减小很多的位姿图

我们知道，在 BA 中特征点数量远大于位姿节点。一个关键帧往往关联了数百个关键点，而实时 BA 的最大计算规模，即使利用稀疏性，在当前的主流 CPU 上一般也就是几万个点左右。这就限制了 SLAM 应用场景。所以，当机器人在更大范围的时间和空间中运动时，必须考虑一些解决方式：要么像滑动窗口法那样，丢弃一些历史数据[92]；要么像位姿图的做法那样，舍弃对路标点的优化，只保留 Pose 之间的边[93–95]。此外，如果我们有额外测量 Pose 的传感器，那么位姿图也是一种常见的融合 Pose 测量的方法。

10.2.2 位姿图的优化

那么，位姿图优化中的节点和边都是什么意思呢？这里的节点表示相机位姿，以 $\boldsymbol{T}_1, \cdots, \boldsymbol{T}_n$ 来表达。而边，则是两个位姿节点之间相对运动的估计，该估计可以来自于特征点法或直接法，也可以来自 GPS 或 IMU 积分。无论通过哪种手段，假设我们估计了 $\boldsymbol{T}_i$ 和 $\boldsymbol{T}_j$ 之间的一个运动

$\Delta \boldsymbol{T}_{ij}$。该运动可以有若干种表达方式，我们取比较自然的一种：

$$\boxed{\Delta \boldsymbol{\xi}_{ij} = \boldsymbol{\xi}_i^{-1} \circ \boldsymbol{\xi}_j = \ln\left(\boldsymbol{T}_i^{-1}\boldsymbol{T}_j\right)^\vee,} \tag{10.2}$$

或按李群的写法：

$$\boldsymbol{T}_{ij} = \boldsymbol{T}_i^{-1}\boldsymbol{T}_j. \tag{10.3}$$

按照图优化的思路，实际中该等式不会精确地成立，因此我们设立最小二乘误差，然后和以往一样，讨论误差关于优化变量的导数。这里，我们把上式的 $\boldsymbol{T}_{ij}$ 移至等式右侧，构建误差 $\boldsymbol{e}_{ij}$：

$$\boldsymbol{e}_{ij} = \Delta \boldsymbol{\xi}_{ij} \ln\left(\boldsymbol{T}_{ij}^{-1}\boldsymbol{T}_i^{-1}\boldsymbol{T}_j\right)^\vee \tag{10.4}$$

若成立，则右侧为0

注意优化变量有两个：$\boldsymbol{\xi}_i$ 和 $\boldsymbol{\xi}_j$，因此我们求 $\boldsymbol{e}_{ij}$ 关于这两个变量的导数。按照李代数的求导方式，给 $\boldsymbol{\xi}_i$ 和 $\boldsymbol{\xi}_j$ 各一个左扰动：$\boldsymbol{\delta\xi}_i$ 和 $\boldsymbol{\delta\xi}_j$。于是误差变为

$$\hat{\boldsymbol{e}}_{ij} = \ln\left(\boldsymbol{T}_{ij}^{-1}\boldsymbol{T}_i^{-1}\exp((-\boldsymbol{\delta\xi}_i)^\wedge)\exp(\delta\boldsymbol{\xi}_j^\wedge)\boldsymbol{T}_j\right)^\vee. \tag{10.5}$$

该式中，两个扰动项被夹在了中间。为了利用 BCH 近似，我们希望把扰动项移至式子左侧或右侧。回忆第 4 讲习题中的伴随性质，即式 (4.55)。如果你没有做过这个习题，那就暂时把它当作是正确的结论来使用：

$$\exp\left((\mathrm{Ad}(\boldsymbol{T})\boldsymbol{\xi})^\wedge\right) = \boldsymbol{T}\exp(\boldsymbol{\xi}^\wedge)\boldsymbol{T}^{-1}. \tag{10.6}$$

稍加改变，有

$$\exp(\boldsymbol{\xi}^\wedge)\boldsymbol{T} = \boldsymbol{T}\exp\left(\left(\mathrm{Ad}(\boldsymbol{T}^{-1})\boldsymbol{\xi}\right)^\wedge\right). \tag{10.7}$$

该式表明，通过引入一个伴随项，我们能够“交换”扰动项左右侧的 $\boldsymbol{T}$。利用它，可以将扰动挪到最右（当然最左亦可），导出右乘形式的雅可比矩阵（挪到左边时形成左乘）：

$$\begin{aligned}
\hat{\boldsymbol{e}}_{ij} &= \ln\left(\boldsymbol{T}_{ij}^{-1}\boldsymbol{T}_i^{-1}\exp((-\boldsymbol{\delta\xi}_i)^\wedge)\exp(\delta\boldsymbol{\xi}_j^\wedge)\boldsymbol{T}_j\right)^\vee \\
&= \ln\left(\boldsymbol{T}_{ij}^{-1}\boldsymbol{T}_i^{-1}\boldsymbol{T}_j\exp\left(\left(-\mathrm{Ad}(\boldsymbol{T}_j^{-1})\boldsymbol{\delta\xi}_i\right)^\wedge\right)\exp\left(\left(\mathrm{Ad}(\boldsymbol{T}_j^{-1})\boldsymbol{\delta\xi}_j\right)^\wedge\right)\right)^\vee \\
&\approx \ln\left(\boldsymbol{T}_{ij}^{-1}\boldsymbol{T}_i^{-1}\boldsymbol{T}_j\left[\boldsymbol{I} - (\mathrm{Ad}(\boldsymbol{T}_j^{-1})\boldsymbol{\delta\xi}_i)^\wedge + (\mathrm{Ad}(\boldsymbol{T}_j^{-1})\boldsymbol{\delta\xi}_j)^\wedge\right]\right)^\vee \\
&\approx \boldsymbol{e}_{ij} + \frac{\partial \boldsymbol{e}_{ij}}{\partial \boldsymbol{\delta\xi}_i}\boldsymbol{\delta\xi}_i + \frac{\partial \boldsymbol{e}_{ij}}{\partial \boldsymbol{\delta\xi}_j}\boldsymbol{\delta\xi}_j
\end{aligned}. \tag{10.8}$$

因此，按照李代数上的求导法则，我们求出了误差关于两个位姿的雅可比矩阵。关于 $\boldsymbol{T}_i$ 的：

$$\frac{\partial \boldsymbol{e}_{ij}}{\partial \delta \boldsymbol{\xi}_i} = -\boldsymbol{\mathcal{J}}_r^{-1}(\boldsymbol{e}_{ij})\mathrm{Ad}(\boldsymbol{T}_j^{-1}). \tag{10.9}$$

以及关于 $\boldsymbol{T}_j$ 的：

$$\frac{\partial \boldsymbol{e}_{ij}}{\partial \delta \boldsymbol{\xi}_j} = \boldsymbol{\mathcal{J}}_r^{-1}(\boldsymbol{e}_{ij})\mathrm{Ad}(\boldsymbol{T}_j^{-1}). \tag{10.10}$$

如果读者觉得这部分求导理解起来有困难，可以回到第 4 讲温习李代数部分的内容。不过，前面也说过，由于 $\mathfrak{se}(3)$ 上的左右雅可比 $\boldsymbol{\mathcal{J}}_r$ 形式过于复杂，我们通常取它们的近似。如果误差接近零，我们就可以设它们近似为 $\boldsymbol{I}$ 或

$$\boldsymbol{\mathcal{J}}_r^{-1}(\boldsymbol{e}_{ij}) \approx \boldsymbol{I} + \frac{1}{2}\begin{bmatrix} \boldsymbol{\phi}_{\boldsymbol{e}}^{\wedge} & \boldsymbol{\rho}_{\boldsymbol{e}}^{\wedge} \\ \boldsymbol{0} & \boldsymbol{\phi}_{\boldsymbol{e}}^{\wedge} \end{bmatrix}. \tag{10.11}$$

理论上，即使在优化之后，由于每条边给定的观测数据并不一致，误差也不见得近似于零，所以简单地把这里的 $\boldsymbol{\mathcal{J}}_r$ 设置为 $\boldsymbol{I}$ 会有一定的损失。稍后我们将通过实践了解理论上的区别是否明显。

了解雅可比求导后，剩下的部分就和普通的图优化一样了。简而言之，所有的位姿顶点和位姿——位姿边构成了一个图优化，本质上是一个最小二乘问题，优化变量为各个顶点的位姿，边来自于位姿观测约束。记 $\mathcal{E}$ 为所有边的集合，那么总体目标函数为

$$\boxed{\min \frac{1}{2}\sum_{i,j\in\mathcal{E}} \boldsymbol{e}_{ij}^{\mathrm{T}}\boldsymbol{\Sigma}_{ij}^{-1}\boldsymbol{e}_{ij}.} \tag{10.12}$$

我们依然可以用高斯牛顿法、列文伯格—马夸尔特方法等求解此问题，除了用李代数表示优化位姿，别的都是相似的。根据先前的经验，可以用 Ceres 或 g2o 进行求解。我们不再讨论优化的详细过程，第 9 讲已经讲清楚了。

10.3 实践：位姿图优化

10.3.1 g2o 原生位姿图

下面演示如何使用 g2o 进行位姿图优化。首先，请读者用 g2o_viewer 打开我们预先生成的仿真位姿图，位于 slambook2/ch10/sphere.g2o 中，如图 10-4 所示。

该位姿图是由 g2o 自带的 create sphere 程序仿真生成的。它的真实轨迹为一个球，由从下往上的多个层组成。每层为一个正圆形，很多个大小不一的圆形层组成了一个完整的球体，共包含 2500 个位姿节点（图 10-4 左上），可以看成一个转圈上升的过程。然后，仿真程序生成了 $t-1$

到 t 时刻的边，称为里程计边。此外，又生成层与层之间的边，称为回环（回环检测算法将在第 11 讲详细介绍）。随后，在每条边上添加观测噪声，并根据里程计边的噪声，重新设置节点的初始值。这样，就得到了带累积误差的位姿图数据（图 10-4 右下）。它局部看起来像球体的一部分，但整体形状与球体相差甚远。现在，我们从这些带噪声的边和节点初始值出发，尝试优化整个位姿图，得到近似真值的数据。

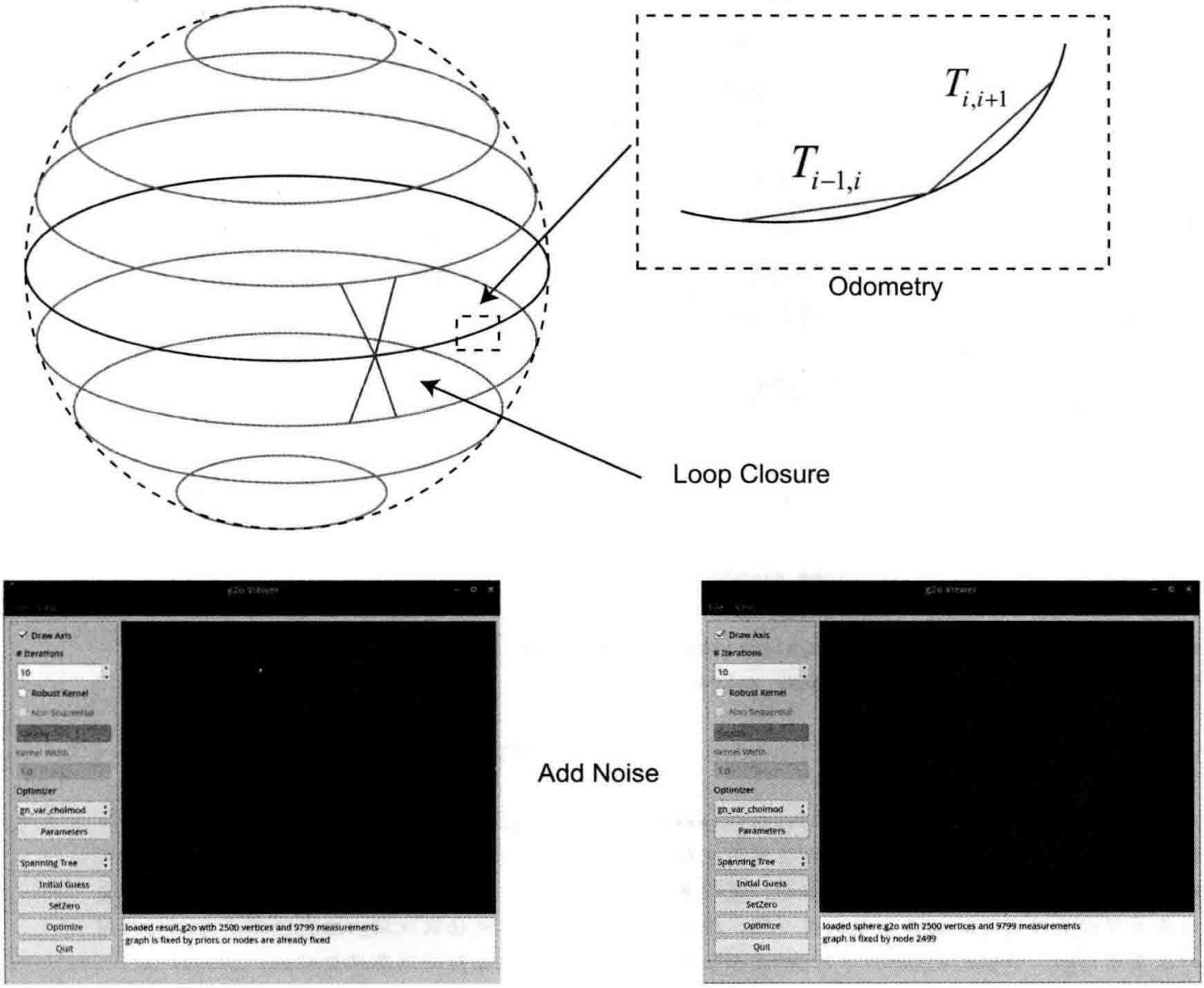

图 10-4　g2o 仿真产生的位姿图。真值是完整的球形，在真值上添加噪声后得到带累计误差的仿真数据

当然，实际中的机器人肯定不会出现这样正球形的运动轨迹，以及如此完整的里程计与回环观测数据。仿真成正球的好处是我们能够直观地看到优化结果是否正确（只要看它各个角度圆不圆就行了）。读者可以单击 g2o_viewer 中的 optimize 函数，看到每步的优化结果和收敛的过程。另外，sphere.g2o 也是一个文本文件，可以用文本编辑器打开，查看它里面的内容。文件前半部

分由节点组成，后半部分则是边：

```
VERTEX_SE3:QUAT 0 -0.125664 -1.53894e-17 99.9999 0.706662 4.32706e-17 0.707551 -4.3325e-17
......
EDGE_SE3:QUAT 1524 1574 -0.210399 -0.0101193 -6.28806 -0.00122939 0.0375067 -2.85291e-05
0.999296 10000 0 0 0 0 0 10000 0 0 0 0 10000 0 0 0 40000 0 0 40000 0 40000
```

可以看到，节点类型是 VERTEX_SE3，表达一个相机位姿。g2o 默认使用四元数和平移向量表达位姿，所以后面的字段意义为：ID，$t_x, t_y, t_z, q_x, q_y, q_z, q_w$。前 3 个为平移向量元素，后 4 个为表示旋转的单位四元数。同样，边的信息为两个节点的 ID，$t_x, t_y, t_z, q_x, q_y, q_z, q_w$，以及信息矩阵的右上角。可以看出，本例中信息矩阵大小为 6×6，且被设成了对角阵。

为了优化该位姿图，我们可以使用 g2o 默认的顶点和边，它们是由四元数表示的。由于仿真数据也是 g2o 生成的，所以用 g2o 本身优化就无须我们多做什么工作了，只需配置优化参数即可。程序 slambook2/ch10/pose_graph_g2o_SE3.cpp 演示了如何使用列文伯格—马夸尔特方法对该位姿图进行优化，并把结果存储至 result.g2o 文件中。

slambook2/ch10/pose_graph_g2o_SE3.cpp

```cpp
#include <iostream>
#include <fstream>
#include <string>

#include <g2o/types/slam3d/types_slam3d.h>
#include <g2o/core/block_solver.h>
#include <g2o/core/optimization_algorithm_levenberg.h>
#include <g2o/solvers/eigen/linear_solver_eigen.h>

using namespace std;

/*************************************************
* 本程序演示如何用g2o solver进行位姿图优化
* sphere.g2o是人工生成的一个位姿图，我们来优化它。
* 尽管可以直接通过load函数读取整个图，但我们还是自己来实现读取代码，以期获得更深刻的理解
* 这里使用g2o/types/slam3d/中的SE3表示位姿，它实质上是四元数而非李代数
* *************************************************/

int main(int argc, char **argv) {
    if (argc != 2) {
        cout << "Usage: pose_graph_g2o_SE3 sphere.g2o" << endl;
        return 1;
    }
    ifstream fin(argv[1]);
    if (!fin) {
```

```
        cout << "file " << argv[1] << " does not exist." << endl;
        return 1;
    }

    // 设定g2o
    typedef g2o::BlockSolver<g2o::BlockSolverTraits<6, 6>> BlockSolverType;
    typedef g2o::LinearSolverEigen<BlockSolverType::PoseMatrixType> LinearSolverType;
    auto solver = new g2o::OptimizationAlgorithmLevenberg(
    g2o::make_unique<BlockSolverType>(g2o::make_unique<LinearSolverType>()));
    g2o::SparseOptimizer optimizer;     // 图模型
    optimizer.setAlgorithm(solver);   // 设置求解器
    optimizer.setVerbose(true);       // 打开调试输出

    int vertexCnt = 0, edgeCnt = 0; // 顶点和边的数量
    while (!fin.eof()) {
        string name;
        fin >> name;
        if (name == "VERTEX_SE3:QUAT") {
            // SE3顶点
            g2o::VertexSE3 *v = new g2o::VertexSE3();
            int index = 0;
            fin >> index;
            v->setId(index);
            v->read(fin);
            optimizer.addVertex(v);
            vertexCnt++;
            if (index == 0)
            v->setFixed(true);
        } else if (name == "EDGE_SE3:QUAT") {
            // SE3-SE3边
            g2o::EdgeSE3 *e = new g2o::EdgeSE3();
            int idx1, idx2;     // 关联的两个顶点
            fin >> idx1 >> idx2;
            e->setId(edgeCnt++);
            e->setVertex(0, optimizer.vertices()[idx1]);
            e->setVertex(1, optimizer.vertices()[idx2]);
            e->read(fin);
            optimizer.addEdge(e);
        }
        if (!fin.good()) break;
    }

    cout << "read total " << vertexCnt << " vertices, " << edgeCnt << " edges." << endl;
```

```

    cout << "optimizing ..." << endl;
    optimizer.initializeOptimization();
    optimizer.optimize(30);

    cout << "saving optimization results ..." << endl;
    optimizer.save("result.g2o");

    return 0;
}
```

我们选择了 6×6 的块求解器，使用列文伯格—马夸尔特下降方式，迭代次数选择 30 次。调用此程序对位姿图进行优化：

终端输入：

```
$ build/pose_graph_g2o_SE3 sphere.g2o
read total 2500 vertices, 9799 edges.
optimizing ...
iteration= 0  chi2= 1023011093.851879 edges= 9799 schur= 0 lambda= 805.622433 levenbergIter= 1
iteration= 1  chi2= 385118688.233188  time= 0.863567 cumTime= 1.71545  edges= 9799 schur= 0 lambda= 537.081622 levenbergIter= 1
iteration= 2  chi2= 166223726.693659  time= 0.861235 cumTime= 2.57668  edges= 9799 schur= 0 lambda= 358.054415 levenbergIter= 1
iteration= 3  chi2= 86610874.269316   time= 0.844105 cumTime= 3.42079  edges= 9799 schur= 0 lambda= 238.702943 levenbergIter= 1
iteration= 4  chi2= 40582782.710190   time= 0.862221 cumTime= 4.28301  edges= 9799 schur= 0 lambda= 159.135295 levenbergIter= 1
......
iteration= 28 chi2= 45095.174398 time= 0.869451 cumTime= 30.0809 edges= 9799 schur= 0 lambda= 0.003127 levenbergIter= 1
iteration= 29 chi2= 44811.248504 time= 1.76326  cumTime= 31.8442 edges= 9799 schur= 0 lambda= 0.003785 levenbergIter= 2
saving optimization results ...
```

然后，用 g2o_viewer 打开 result.g2o 查看结果，如图 10-5 所示。

结果从一个不规则的形状优化成了一个看起来完整的球。这个过程实质上和我们单击 g2o_viewer 上的 Optimize 按钮没有区别。下面，我们根据前面的李代数推导来实现李代数上的优化。

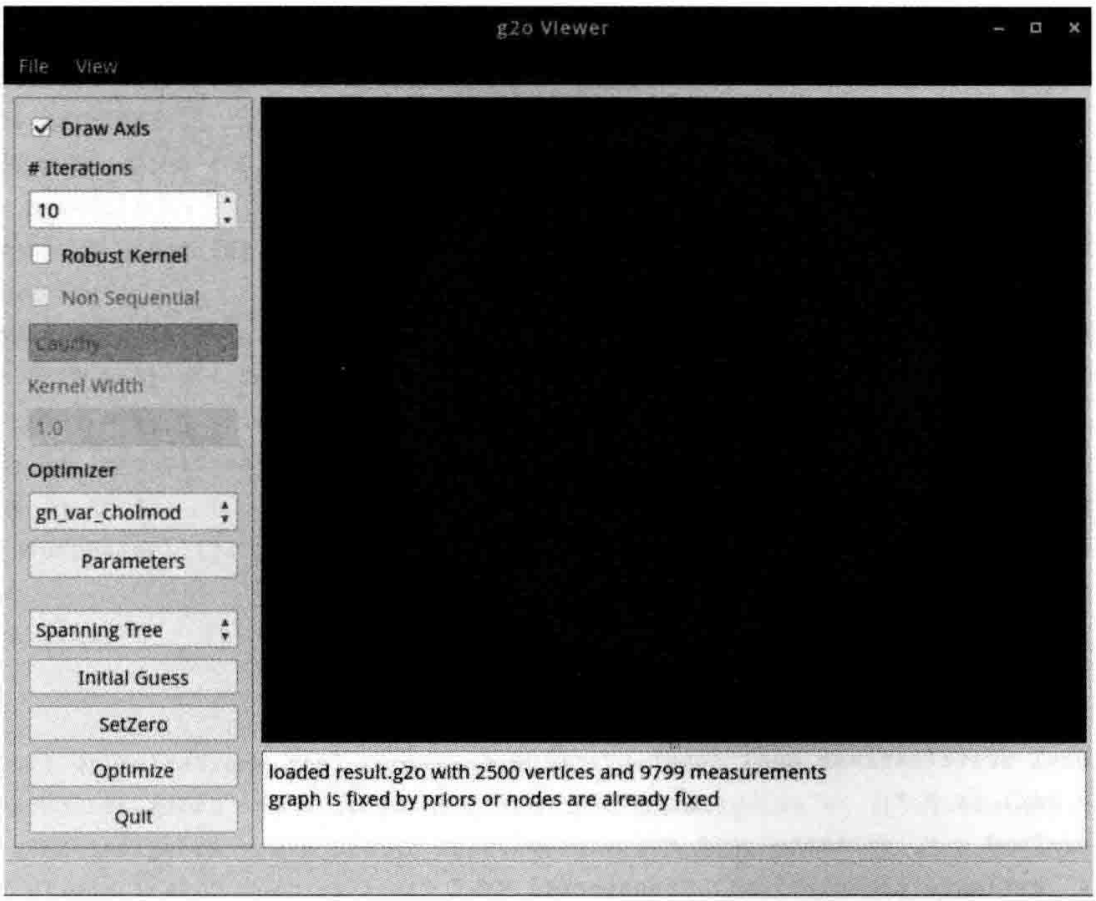

图 10-5　使用 g2o 自带的顶点与边求解的结果

10.3.2　李代数上的位姿图优化

还记得我们用 Sophus 表达李代数的事情吗？我们来试试把 Sophus 用到 g2o 中定义自己的顶点和边吧。

slambook2/ch10/pose_graph_g2o_lie_algebra.cpp（片段）

```
typedef Matrix<double, 6, 6> Matrix6d;

// 给定误差求J_R^{-1}的近似
Matrix6d JRInv(const SE3d &e) {
    Matrix6d J;
    J.block(0, 0, 3, 3) = SO3d::hat(e.so3().log());
    J.block(0, 3, 3, 3) = SO3d::hat(e.translation());
    J.block(3, 0, 3, 3) = Matrix3d::Zero(3, 3);
    J.block(3, 3, 3, 3) = SO3d::hat(e.so3().log());
    J = J * 0.5 + Matrix6d::Identity();
    return J;
}

```

```
// 李代数顶点
typedef Matrix<double, 6, 1> Vector6d;

class VertexSE3LieAlgebra : public g2o::BaseVertex<6, SE3d> {
    public:
    EIGEN_MAKE_ALIGNED_OPERATOR_NEW

    virtual bool read(istream &is) override {
        double data[7];
        for (int i = 0; i < 7; i++)
        is >> data[i];
        setEstimate(SE3d(
        Quaterniond(data[6], data[3], data[4], data[5]),
        Vector3d(data[0], data[1], data[2])
        ));
    }

    virtual bool write(ostream &os) const override {
        os << id() << " ";
        Quaterniond q = _estimate.unit_quaternion();
        os << _estimate.translation().transpose() << " ";
        os << q.coeffs()[0] << " " << q.coeffs()[1] << " " << q.coeffs()[2] << " " << q.
        coeffs()[3] << endl;
        return true;
    }

    virtual void setToOriginImpl() override {
        _estimate = SE3d();
    }

    // 左乘更新
    virtual void oplusImpl(const double *update) override {
        Vector6d upd;
        upd << update[0], update[1], update[2], update[3], update[4], update[5];
        _estimate = SE3d::exp(upd) * _estimate;
    }
};

// 两个李代数节点之边
class EdgeSE3LieAlgebra : public g2o::BaseBinaryEdge<6, SE3d, VertexSE3LieAlgebra,
VertexSE3LieAlgebra> {
    public:
    EIGEN_MAKE_ALIGNED_OPERATOR_NEW
```

```
    virtual bool read(istream &is) override {
        double data[7];
        for (int i = 0; i < 7; i++)
        is >> data[i];
        Quaterniond q(data[6], data[3], data[4], data[5]);
        q.normalize();
        setMeasurement(SE3d(q, Vector3d(data[0], data[1], data[2])));
        for (int i = 0; i < information().rows() && is.good(); i++)
        for (int j = i; j < information().cols() && is.good(); j++) {
            is >> information()(i, j);
            if (i != j)
            information()(j, i) = information()(i, j);
        }
        return true;
    }

    virtual bool write(ostream &os) const override {
        VertexSE3LieAlgebra *v1 = static_cast<VertexSE3LieAlgebra *> (_vertices[0]);
        VertexSE3LieAlgebra *v2 = static_cast<VertexSE3LieAlgebra *> (_vertices[1]);
        os << v1->id() << " " << v2->id() << " ";
        SE3d m = _measurement;
        Eigen::Quaterniond q = m.unit_quaternion();
        os << m.translation().transpose() << " ";
        os << q.coeffs()[0] << " " << q.coeffs()[1] << " " << q.coeffs()[2] << " " << q.
        coeffs()[3] << " ";

        // information matrix
        for (int i = 0; i < information().rows(); i++)
        for (int j = i; j < information().cols(); j++) {
            os << information()(i, j) << " ";
        }
        os << endl;
        return true;
    }

    // 误差计算与书中推导一致
    virtual void computeError() override {
        SE3d v1 = (static_cast<VertexSE3LieAlgebra *> (_vertices[0]))->estimate();
        SE3d v2 = (static_cast<VertexSE3LieAlgebra *> (_vertices[1]))->estimate();
        _error = (_measurement.inverse() * v1.inverse() * v2).log();
    }

```

```
    // 雅可比计算
    virtual void linearizeOplus() override {
        SE3d v1 = (static_cast<VertexSE3LieAlgebra *> (_vertices[0]))->estimate();
        SE3d v2 = (static_cast<VertexSE3LieAlgebra *> (_vertices[1]))->estimate();
        Matrix6d J = JRInv(SE3d::exp(_error));
        // 尝试把J近似为I?
        _jacobianOplusXi = -J * v2.inverse().Adj();
        _jacobianOplusXj = J * v2.inverse().Adj();
    }
};
```

为了实现对 g2o 文件的存储和读取，本节例程实现了 read 和 write 函数，并且“伪装”成 g2o 内置的 SE3 顶点，使得 g2o_viewer 能够认识并渲染它。事实上，除了内部使用 Sophus 的李代数表示，从外部看起来没有什么区别。

值得注意的是这里雅可比的计算过程。我们有若干种选择：一是不提供雅可比计算函数，让 g2o 自动计算数值雅可比。二是提供完整或近似的雅可比计算过程。这里，我们用 JRInv() 函数提供近似的 $\mathcal{J}_r^{-1}$。读者可以尝试把它近似为 $\boldsymbol{I}$，或者干脆注释掉 oplusImpl 函数，看看结果会有什么区别。

之后调用 g2o 进行优化问题：

终端输入：

```
$ build/pose_graph_g2o_lie sphere.g2o
read total 2500 vertices, 9799 edges.
optimizing ...
iteration= 0     chi2= 626657736.014949  time= 0.549125  cumTime= 0.549125   edges= 9799
schur= 0    lambda= 6706.585223     levenbergIter= 1
iteration= 1     chi2= 233236853.521434  time= 0.510685  cumTime= 1.05981    edges= 9799
schur= 0    lambda= 2235.528408     levenbergIter= 1
iteration= 2     chi2= 142629876.750105  time= 0.557893  cumTime= 1.6177     edges= 9799
schur= 0    lambda= 745.176136  levenbergIter= 1
iteration= 3     chi2= 84218288.615592   time= 0.525079  cumTime= 2.14278    edges= 9799
schur= 0    lambda= 248.392045  levenbergIter= 1
......
```

我们发现，迭代 23 次后，总体误差保持不变，事实上可以让优化算法停止。而上一个实验中，用满了 30 次迭代后误差仍在下降[①]。在调用优化后，查看 result_lie.g2o 观察它的结果，如图 10-6 所示。从肉眼上看不出任何区别。

如果你在这个 g2o_viewer 界面单击“Optimize”按钮，g2o 将使用它自带的 SE3 顶点进行优

①请注意，尽管数值上看此处的误差要大一些，但是由于我们自定义边时重新定义了误差的计算方式，所以此处数值的大小并不能直接用于比较。

化，你可以在下方文本框中看到：

```
loaded result_lie.g2o with 2500 vertices and 9799 measurements
graph is fixed by node 2499
# Using CHOLMOD poseDim -1 landMarkDim -1 blockordering 0
Preparing (no marginalization of Landmarks)
iteration= 0 chi2= 44360.509723 time= 0.567504 cumTime= 0.567504 edges= 9799 schur= 0
iteration= 1 chi2= 44360.471110 time= 0.595993 cumTime= 1.1635   edges= 9799 schur= 0
iteration= 2 chi2= 44360.471110 time= 0.582909 cumTime= 1.74641  edges= 9799 schur= 0
```

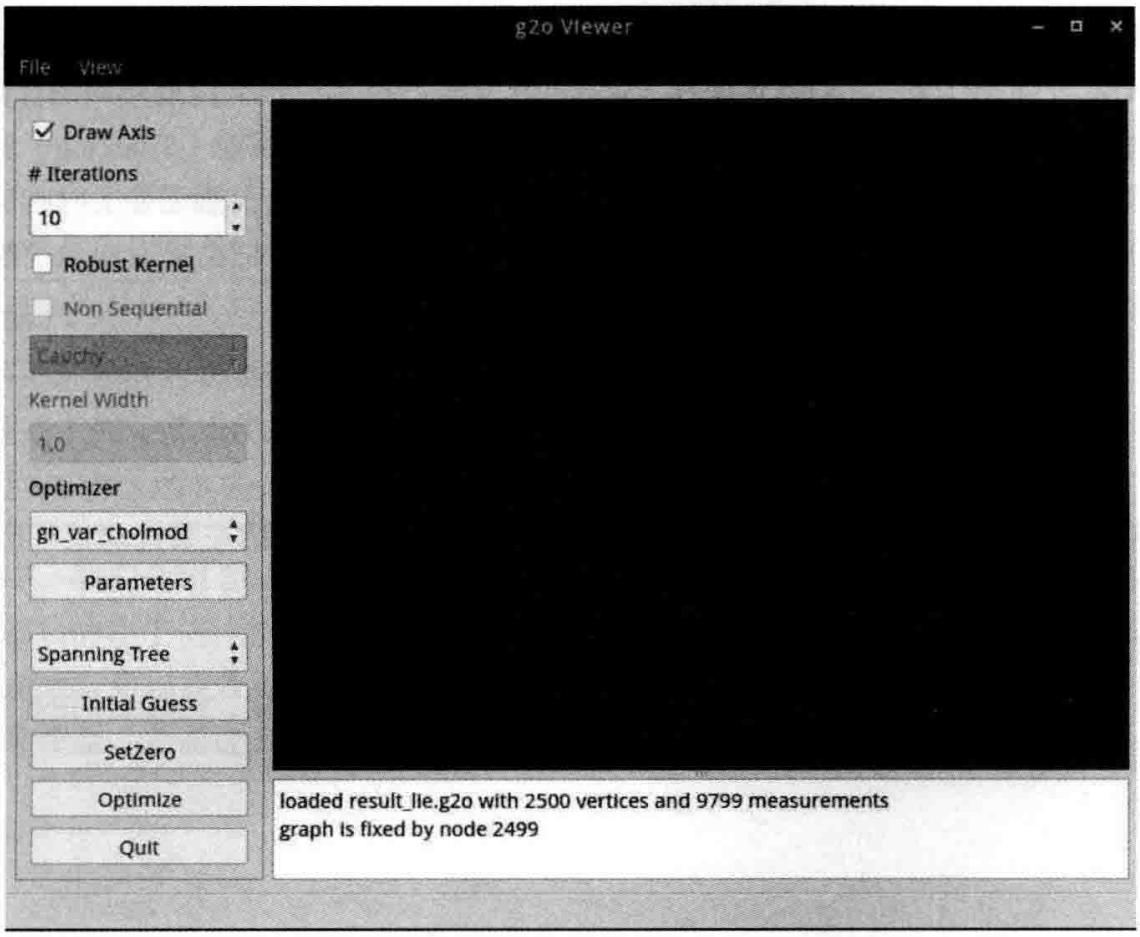

图 10-6　使用李代数自定义节点与边优化后的结果

整体误差在 SE3 边的度量下为 44360，略小于之前 30 次迭代时的 44811。这说明使用李代数进行优化后，我们在更少的迭代次数下得到了更好的结果[①]。实际上，即使我们用单位矩阵来近似 $\mathcal{J}_r^{-1}$，你也会收敛到类似的值。这主要是因为在误差接近零时，雅可比本来就和恒等矩阵十分接近。

①由于没有做更多的实验，所以该结论只在“球”这个例子上有效。

10.3.3 小结

球的例子是一个比较有代表性的案例。它具有和实际中相似的里程计边和回环边，这也正是实际 SLAM 中一个位姿图中可能有的东西。同时，“球”也具有一定的计算规模：一方面，它总共有 2,500 个位姿节点和近 10,000 条边，我们发现优化它花费了不少时间（相对于实时性要求很强的前端来说）。另一方面，一般认为位姿图是结构最简单的图之一。在我们不假设机器人如何运动的前提下，很难进一步讨论它的稀疏性——因为机器人可能会直线往前运动，形成带状的位姿图，是稀疏的；也可能是“左手右手一个慢动作”，形成来回往复的循环运动（Loopy motion），从而变成像“球”那样比较稠密的位姿图。无论如何，在没有进一步的信息之前，我们似乎无法再利用位姿图的求解结构了。

自 PTAM[96] 提出以来，人们就意识到，后端的优化没必要实时地响应前端的图像数据。人们倾向于把前端和后端分开，运行于两个独立线程之中，历史上称为跟踪（Tracking）和建图——虽然如此称呼，但是建图部分主要是指后端的优化内容。通俗地说，前端需要实时响应视频的速度，例如每秒 30 帧；而优化可以慢悠悠地运行，只要在优化完成时把结果返回给前端即可。所以我们通常不会对后端优化提出很高的速度要求。

习题

1. 将位姿图中的误差定义为 $\Delta\boldsymbol{\xi}_{ij} = \boldsymbol{\xi}_i \circ \boldsymbol{\xi}_j^{-1}$，推导按照此定义的左乘扰动雅可比矩阵。
2. 使用右乘更新，推导该情况下的雅可比矩阵。
3. 参照 g2o 的程序，在 Ceres 中实现对“球”位姿图的优化。
4. 对“球”中的信息按照时间排序，分别喂给 g2o 和 gtsam 优化，比较它们的性能差异。
5.* 阅读与 iSAM 相关的论文，理解它是如何实现增量式优化的。